

Kapsülleme ve Çok Biçimlilik

NTP II - Hafta 5



Nesne Tabanlı Programlamada Uсталık: Kapsülleme ve Çok Biçimlilik

Bu sunum, Nesne Tabanlı Programlama (NTP) ilkelerinin iki temel diređi olan Kapsülleme ve Çok Biçimlilik konularını derinlemesine incelemektedir. Bu prensipler, sadece kod organizasyonunu deđil, aynı zamanda yazılımın güvenliđini ve sürdürülebilirliđini de doğrudan etkiler. Üniversite düzeyindeki bu derste, konuların akademik ve pratik uygulamalarını ele alacağız.



Nesne Tabanlı Programlama (NTP) Temellerine Hızlı Bakış

NTP, gerçek dünyadaki varlıkları yazılım nesneleri olarak modellemeyi amaçlayan bir paradigmaları bütünüdür.

Temel taşları şunlardır:

1. Soyutlama (Abstraction): Karmaşık detayları gizleme.
2. Kapsülleme (Encapsulation): Veri ve metotları bir araya getirme ve koruma.
3. Kalıtım (Inheritance): Yeni sınıflar oluşturmak için mevcut sınıfların özelliklerini devralma.
4. Çok Biçimlilik (Polymorphism): Farklı nesnelerin aynı mesaja farklı yanıtlar verebilmesi.

Kapsülleme Nedir? (Encapsulation)

Kapsülleme, bir nesnenin verilerini (niteliklerini) ve bu veriler üzerinde çalışan metotları (davranışlarını) tek bir birim içinde birleştirme sürecidir.

Aynı zamanda, verinin dış dünyadan korunması, yani 'veri gizleme' (Data Hiding) ilkesini içerir.

Bu, bir sınıfın dahili çalışma mekanizmasının dışarıdan erişilemez veya değiştirilemez olmasını sağlar.



Kapsüllemenin Temel Amacı

Kapsüllemenin iki ana amacı vardır:

1. Veri Bütünlüğünü Koruma: Verilere doğrudan erişimi engelleyerek, verinin geçerli olmayan bir durumda kalmasını önlemek.
2. Gevşek Bağlılık (Loose Coupling) Sağlama: Bir nesnenin dahili uygulamasını değiştirdiğinizde, bunu kullanan diğer kodların etkilenme riskini azaltmak (Modülerlik).

Kapsülleme ve Soyutlama Arasındaki İlişki

Soyutlama, neyin yapıldığını gösterirken (fonksiyonellik), Kapsülleme bunu nasıl yaptığımızı gizler (uygulama detayları).

Kapsülleme, Soyutlamanın pratik olarak uygulanmasına yardımcı olan bir mekanizmadır.

Örneğin, bir arabanın gaz pedalına basmak Soyutlamadır; gaz pedalının motorla nasıl iletişim kurduğu Kapsüllemedir.



Erişim Belirleyicileri (Access Modifiers)

NTP dillerinde (Java, C#, Python vb.), bir özelliğin veya metodun nerede ve kim tarafından kullanılabileceğini kontrol eden belirleyiciler kullanılır.

Ana belirleyiciler:

1. Public: Her yerden erişilebilir.
2. Protected: Sadece aynı sınıf veya türetilmiş alt sınıflar tarafından erişilebilir.
3. Private: Sadece tanımlandığı sınıf içerisinden erişilebilir (veri gizlemenin temel aracıdır).



Public Eriřim Seviyesi

Public üyeler, nesnenin dış arayüzünü oluşturur.
Bu üyelere, sınıfın dışındaki herhangi bir kod parçası tarafından doğrudan erişilebilir ve değiştirilebilir.
Genellikle, nesnenin dış dünyaya sunmak istediğı temel davranışlar (metotlar) bu seviyede tanımlanır.



Private Eriřim Seviyesi

Private üyeler, sınıfın dahili çalışma detaylarını oluşturur.
Bu üyelere, sadece o sınıfın kendi metotları içerisinde erişim sağlanabilir.
Veri bütünlüğünü korumak için, kritik nitelikler daima private olarak tanımlanmalıdır.



Protected Eriřim Seviyesi

Protected üyeler, veri gizleme ile esneklik arasında bir denge sunar. Bu üyeler, tanımlandıkları sınıf ve bu sınıftan türetilen alt sınıflar tarafından erişilebilir. Bu, ana sınıfın bazı dahili özelliklerinin alt sınıflar tarafından özelleştirilebilmesine olanak tanır, ancak dış dünyadan korunur.



Python'da Erişim Belirleyiciler

Python'da katı 'private' anahtar kelimesi yoktur. Erişim, adlandırma kuralları (naming conventions) ile sağlanır:

1. Public: Normal adlandırma (örn: 'nitelik').
2. Protected: Tek alt çizgi ile başlar (örn: '_protected_nitelik'). Bu, dışarıdan erişilmemesi gerektiğini belirten bir sözleşmedir.
3. Private: İki alt çizgi ile başlar (örn: '__private_nitelik'). Bu, isimlendirme karışıklığı (name mangling) ile erişimi zorlaştırır.

Getter Metotları (Accessors)

Getter metotları, private olarak tanımlanmış bir niteliğin değerini okumak için kullanılır.

Bu metotlar, dış dünyaya veriyi sunarken herhangi bir kısıtlama veya işleme yapma ihtiyacı duyulmadığında faydalıdır.

Erişimin kontrollü ve read-only (sadece okunabilir) olmasını sağlar.



Setter Metotları (Mutators)

Setter metotları, private olarak tanımlanmış bir niteliğin değerini değiştirmek için kullanılır.

Bu metotlar, en kritik kapsülleme mekanizmalarından biridir, çünkü değer atamadan önce doğrulama (validation) ve temizleme (sanitization) işlemleri yapılmasına olanak tanır.

Örneğin, yaş değerinin negatif olmamasını veya notun 100'den büyük olmamasını kontrol etmek.



Kapsülleme Örneği: Öğrenci Sınıfı

Senaryo: Bir 'Öğrenci' sınıfındaki 'ogrenci_no' ve 'not_ortalaması' niteliklerini private yapın.

Amaç: Bu niteliklere doğrudan erişimi engelleyerek, dışarıdan yanlış veri girişi yapılmasını önlemek.

Sadece getter ve setter metotları ile erişim sağlanmalıdır.

Ogrenci Sınıfı Setter Uygulaması

Setter metodu ('set_not_ortalaması') ile kural tanımlanır:

Eğer yeni not ortalaması 0 ile 4.0 arasında değilse, atama yapılmaz veya bir hata fırlatılır.

Bu, sınıfın dahili durumunun her zaman geçerli (consistent) kalmasını garanti eder.



Veri Gizlemenin Güvenlik Katkısı (I)

Kapsülleme, nesnenin dahili verilerinin dış müdahalelerden nasıl korunduğunu gösterir.

Eğer veriye doğrudan erişim olsaydı, kötü niyetli bir kullanıcı veya hatalı bir kod parçası, önemli sistem değişkenlerini geçersiz değerlere ayarlayabilirdi. Private değişkenler bu doğrudan erişimi engeller.



Veri Gizlemenin Güvenlik Katkısı (II)

Kapsülleme sayesinde, bazı kritik nitelikleri sadece getter metodu sağlayarak değiştirilemez (immutable) yapabiliriz.

Değiştirilemez nesneler, çoklu iş parçacığı (multi-threading) ortamlarında ve güvenlik gerektiren uygulamalarda hataların ve yarış koşullarının (race conditions) önlenmesi açısından hayati önem taşır.



Kapsülleme, veriyi değiştirme mantığını tek bir yerde (setter içinde) merkezileştirir.

Bu sayede, iş kuralı değişiklikleri tek bir noktada yapılabilir ve kodun tamamına yayılmaz.

Bu, yazılımın sürdürülebilirliğini ve güvenilirliğini artırır.



Sıkça Yapılan Hata: Setter Olmadan Public Eriřim

Eğer nitelikler public bırakılırsa veya setter metotları doğrulama yapmazsa, sınıfın dahili durumu her an tehlikeye girebilir.

Bu, özellikle büyük projelerde hata ayıklamayı zorlaştırır ve sistemin öngörülebilirliğini azaltır.

Kapsülleme olmadan NTP'nin faydalarından tam olarak yararlanılamaz.



Kapsülleme ve Arabirimler (Interfaces)

Arabirimler, bir sınıfın hangi metotları uygulaması gerektiğini belirten sözleşmelerdir.

Arabirimler kullanılarak, kullanıcıya sadece yapılması gerekenler gösterilirken, arkadaki karmaşık uygulama detayları kapsülленir.

Bu, Mimaride Bağıllığı Azaltır (Decoupling).

Kapsülleme Özeti

Private nitelikler kullanmak ve bunlara Getter/Setter üzerinden erişmek:

1. Veri Bütünlüğü Garanti Edilir: Geçersiz veri girişi engellenir.
2. Esneklik Sağlanır: Dahili veri yapısını dış kodu etkilemeden değiştirebiliriz.
3. Güvenlik Artar: Doğrudan manipülasyon önlenir ve kurallara uygun erişim zorlanır.

Çok Biçimlilik Nedir? (Polymorphism)

Çok biçimlilik kelime anlamı olarak 'birçok şekil' demektir. NTP bağlamında, farklı nesnelerin aynı arayüze (metot imzasına) sahip olması, ancak bu arayüze farklı şekillerde yanıt vermesi yeteneğidir. Bu, kodun daha esnek, genişletilebilir ve yeniden kullanılabilir olmasını sağlar.

Çok Biçimliliğin Türleri

Çok Biçimlilik genellikle iki ana kategoride incelenir:

1. Derleme Zamanı (Static) Çok Biçimlilik: Genellikle Metot Aşırı Yükleme (Method Overloading) ile sağlanır.
2. Çalışma Zamanı (Dynamic) Çok Biçimlilik: Genellikle Metot Ezme (Method Overriding) ve Kalıtım ile sağlanır.

Derleme Zamanı Çok Biçimlilik: Metot Aşırı Yükleme

Metot aşırı yükleme (Method Overloading), bir sınıfta aynı isme sahip birden fazla metot tanımlanmasıdır.

Ancak, bu metotların parametre listeleri (sayısı veya türü) farklı olmalıdır. Derleyici, hangi metodun çağrılacağına derleme zamanında (statik olarak) karar verir.



Çalışma Zamanı Çok Biçimlilik: Metot Ezme

Çalışma zamanı çok biçimlilik, bir alt sınıfın, üst sınıftan miras aldığı bir metodu kendi ihtiyacına göre yeniden uygulamasıdır (override etmesidir).
Bu durumda, derleme zamanında üst sınıf metodu çağrılıyor gibi görünse de, çalışan nesnenin tipine göre (dinamik olarak) alt sınıf metodu çağrılır.

Ezme (Overriding) İçin Ön Koşullar

Bir metodu doğru bir şekilde ezme için aşağıdaki koşullar sağlanmalıdır:

1. Metot, bir üst sınıftan miras alınmış olmalıdır.
2. Metodun ismi, dönüş tipi ve parametre listesi (imzası) üst sınıftaki metot ile tamamen aynı olmalıdır.
3. Erişim belirleyici, üst sınıftaki metottan daha kısıtlayıcı olamaz (örneğin public metodu private yapamazsınız).

Çalışma Zamanı Bağlama (Runtime Binding)

Çok biçimliliğin en kritik mekanizması dinamik bağlamadır (Dynamic Binding). Bu, programın hangi metodu çağıracağına derleme zamanında değil, çalışma zamanında (nesnenin gerçek tipine bakarak) karar vermesi demektir. Bu esneklik, kodu somut sınıflara bağımlı olmaktan kurtarır.

Uygulama Örneği: Hayvan Sınıfları

Bir 'Hayvan' üst sınıfı ve ondan türetilen 'Kedi' ve 'Kopek' alt sınıflarını düşünün. 'Hayvan' sınıfında genel bir 'konus()' metodu tanımlanır. 'Kedi' sınıfı bu metodu 'Miyav' demek için, 'Kopek' sınıfı ise 'Hav Hav' demek için ezer (override eder).

Soyut Sınıflar ve Çok Biçimlilik

Soyut (Abstract) sınıflar, alt sınıfların mutlaka uygulaması gereken soyut metotlar içerebilir.

Bu soyut metotlar, çok biçimliliğin sağlanması için temel bir arayüz tanımlar. Soyut sınıflardan nesne türetilemez; sadece kalıtım yoluyla kullanılabilirler.



Arabirimler ve Çok Biçimlilik

Arabirimler (Interfaces), sadece metot imzalarını (sözleşmeleri) tanımlar, uygulama detaylarını içermez.

Bir sınıf birden fazla arabirim uygulayabilir.

Bu, farklı sınıflardan gelen nesnelerin bile aynı şekilde ele alınmasını (ortak bir arabirim üzerinden) sağlayarak çok biçimliliği destekler.



Çok Biçimliliğin Faydaları (I)

Çok biçimlilik, sisteme yeni tipler (sınıflar) eklemeyi mevcut kodu değiştirmeden mümkün kılar.

Örneğin, 'Hayvan' listesini işleyen bir metot, sonradan eklenen 'Kus' sınıfını da herhangi bir kod değişikliği olmadan işleyebilir.

Bu, yazılımın gelecekteki değişikliklere daha dayanıklı olmasını sağlar.



Çok Biçimliliğin Faydaları (II)

Kod, genel tipler (üst sınıflar veya arayüzler) üzerinden yazıldığı için, özel (somut) sınıflar için ayrı ayrı kod yazma ihtiyacı azalır. Bu sayede daha az tekrar eden (DRY - Don't Repeat Yourself) ve daha okunabilir kod elde edilir.



Upcasting ve Downcasting

Upcasting (Yukarı Tip Dönüşümü): Bir alt sınıf nesnesinin üst sınıf referansına atanmasıdır. Bu, güvenlidir ve çok biçimliliğin temelini oluşturur.

Downcasting (Aşağı Tip Dönüşümü): Bir üst sınıf referansının alt sınıf tipine dönüştürülmesidir. Bu, potansiyel olarak tehlikelidir ve çalışma zamanı hatalarına yol açabilir, dikkatli kullanılmalıdır.

Geç Bağlama (Late Binding) Detayları

C++ ve benzeri dillerde dinamik çok biçimlilik, sanal tablo (Virtual Table veya V-Table) yapısı ile sağlanır.

Bir sınıfta ezilebilen (virtual) metotlar varsa, bu sınıfa ait her nesne, hangi metodun çağrılacağını belirten bir işaretçiye (vptr) sahiptir.

Bu tablo, çalışma zamanında doğru metodu bulmayı sağlar.

Kapsülleme ve Çok Biçimliliğin Sinerjisi

Bu iki ilke birbirini tamamlar:

Kapsülleme, nesnenin durumunun (verisinin) dahili ve güvenli kalmasını sağlar.

Çok Biçimlilik, nesnenin davranışının (metotlarının) dışarıdan esnek bir şekilde kullanılabilmesini sağlar.

Birlikte, sağlam ve adapte edilebilir bir yazılım mimarisi oluştururlar.



Bu iki temel prensip, modern yazılım tasarımının omurgasını oluşturan SOLID ilkelerini doğrudan destekler:

S (Single Responsibility Principle) Tek Sorumluluk Prensibi: Kapsülleme ile desteklenir.

O (Open/Closed Principle) Açık/Kapalı Prensibi: Çok biçimlilik (özellikle ezme) ile desteklenir. (Değişikliğe kapalı, genişletilmeye açık).

Bilişim Güvenliği Açısından Kapsülleme

Kapsülleme, sistem güvenliğinin temelini oluşturur:

1. Oturum Bilgileri: Kullanıcının kimlik doğrulama belirteçleri (tokenlar) gibi kritik oturum bilgileri private nitelikler olarak tutulmalıdır.
2. Yetkisiz Erişim Önleme: Setter metotları, bir kullanıcıya atanacak yetki seviyesi gibi güvenlik açısından hassas değerlerin yalnızca geçerli kontrol mekanizmalarından geçerek değiştirilmesini sağlar.

Güvenlik Açısından Çok Biçimlilik

Çok biçimlilik, farklı kullanıcı rollerine (Yönetici, Müşteri, Misafir) sahip nesnelerin aynı 'erisim_kontrol()' metodunu farklı şekillerde uygulamasına olanak tanır.

Bu sayede, güvenlik mekanizması (yetkilendirme katmanı) esnek bir şekilde genişletilebilirken, ana arayüz (metot imzası) sabit kalır.



Veri Bütünlüğü İçin Hash ve Şifreleme

Bir 'Kullanici' sınıfında, parolalar açık metin olarak tutulmamalıdır. Kapsülleme (private nitelikler ve setterlar) bu zorunluluğu destekler: Setter metodu, kullanıcı parolayı girdiğinde, parolayı kaydetmeden önce güvenli bir hash algoritması (SHA-256 gibi) kullanarak onu dönüştürür.

Tasarım Kalıpları ve Kapsülleme: Fabrika Metodu

Fabrika Metodu (Factory Method) tasarım kalıbı, nesne oluşturma mantığını kapsüller.

Kullanıcı, hangi somut sınıfın oluşturulduğunu bilmeden, sadece bir soyut arayüz (Fabrika) aracılığıyla nesne talep eder.

Bu, bağımlılıkları azaltır ve kodun karmaşıklığını gizler.



Tasarım Kalıpları ve Çok Biçimlilik: Strateji Kalıbı

Strateji (Strategy) tasarım kalıbı, çok biçimliliği kullanarak, bir nesnenin davranışını çalışma zamanında değiştirmeyi sağlar. Farklı algoritmalar (stratejiler) aynı arabirimi uygular. Ana sınıf, hangi stratejiyi kullanacağını bilmeden, sadece arabirim üzerinden 'calistir()' metodunu çağırır.



İstisna Yönetiminde Kapsülleme

Kapsülleme, dahili sistem hatalarının (veritabanı bağlantı detayları gibi) dış dünyaya sızmasını engeller.

Hata oluştuğunda, private kalması gereken detaylar yerine, kullanıcıya sadece genel ve güvenli bir hata mesajı gönderilmelidir.

Bu, potansiyel güvenlik açıklarını (örneğin bilgi sızması) önler.



Sözleşmeye Göre Programlama (Design by Contract)

Sözleşmeye Göre Programlama, metotların ön koşullarını (preconditions) ve son koşullarını (postconditions) tanımlamayı gerektirir. Setter metotları, ön koşulları (veri doğrulama kurallarını) uygulamak için ideal yerlerdir. Getter metotları ise, nesnenin durumunun her zaman geçerli bir son koşulu temsil ettiğini varsayar.

Refactoring (Yeniden Düzenleme) ve Kapsülleme

Kapsüllenmiş bir sınıfta, eğer dahili veri yapısını (örneğin bir 'List' yerine 'Set' kullanmaya başlamak) değiştirirsek, bu değişiklik dış arayüze (getter/setter metot imzalarına) yansımaz.

Bu, büyük sistemlerde yeniden düzenleme maliyetlerini ve risklerini minimize eder.



Çok Biçimlilik ve Standart Kütüphaneler

NTP dillerindeki koleksiyon çatısı (List, Map, Queue), çok biçimliliğin güçlü bir uygulamasıdır.

Örneğin, bir 'List<Hayvan>' tanımladığınızda, bu liste hem 'Kedi' hem de 'Kopek' nesnelerini saklayabilir ve her birine aynı 'konus()' metodunu çağırabilir. Bu, kütüphane kullanımını basitleştirir.



Aşırı Yükleme mi, Ezme mi? (Overloading vs. Overriding)

Metot Aşırı Yükleme: Aynı isim, farklı imzalar. Derleme zamanı çözülür. Tek bir sınıf içinde olabilir.

Metot Ezme: Aynı isim, aynı imza. Çalışma zamanı çözülür. Kalıtım zinciri gerektirir.

Her ikisi de çok biçimlilik sağlamasına rağmen, amaçları ve teknik uygulamaları farklıdır.



Kapsüllemenin Anti-Kalıbı (Anti-Pattern): God Object

God Object (Tanrı Nesnesi), neredeyse tüm veriyi ve mantığı kapsülleyen, ancak çok fazla sorumluluk yüklenen sınıflardır. Bu, kapsüllemenin yanlış kullanımıdır, çünkü sınıfın iç işleyişi o kadar karmaşıklaşır ki, gevşek bağıllık ilkesi kaybolur ve bakım imkansız hale gelir. Kapsülleme, Tek Sorumluluk Prensipleri ile birlikte kullanılmalıdır.

Programlamada Güvenlik Katmanı Olarak Kapsülleme

Kapsülleme, erişim kontrol listeleri (ACL) veya rol tabanlı erişim kontrolü (RBAC) gibi güvenlik mekanizmalarını uygulamada kullanılır. Private veriler, sadece yetkilendirilmiş metotlar (yani getter/setterlar) aracılığıyla değiştirilebilir. Bu, bir tür 'kapalı kutu' güvenlik modeli oluşturur.



Polymorphism ile Bağımlılık Enjeksiyonu (DI)

Bağımlılık Enjeksiyonu, bir bileşenin bağımlılıklarını çalışma zamanında almasını sağlayan bir tekniktir.

Bu teknik, tamamen çok biçimliliğe dayanır (arayüzler ve ezilmiş metotlar). Sistem bileşenlerini somut sınıflara bağımlı olmaktan kurtarır ve test edilebilirliği (unit testing) artırır.



Getter/Setter Yerine Özellikler (Properties)

C# ve Python gibi modern diller, getter/setter metotlarını yazma zorunluluğunu azaltmak için 'özellikler' (properties) kavramını sunar. Bu mekanizma, kullanıcıya niteliğe doğrudan erişiyormuş gibi görünürken, arka planda otomatik olarak set ve get metotlarını çağırır. Bu, kodun daha temiz ve kısa olmasını sağlar.



Örnek Senaryo: Veri Akışı Kontrolü

Finansal bir uygulamada, bir hesabın bakiyesi private olmalıdır. Para yatırma/çekme işlemleri setter metotları üzerinden yapılmalıdır. Setter, bakiye sıfırın altına düştüğünde (ön koşul ihlali) işlemi reddetmeli veya bir istisna fırlatmalıdır. Bu, veri akışını ve hesap durumunu sürekli kontrol altında tutar.



Dinamik Programlamanın Gücü

Çok biçimlilik, yazılım mimarisini daha dinamik hale getirir. Eklentilerin (plugins) veya modüllerin sisteme çalışma zamanında yüklenip, tanımlanmış bir arayüz üzerinden ana sistem tarafından kullanılabilmesine olanak tanır. Bu, Microservices mimarisi gibi dağıtık sistemler için hayati önem taşır.



Veri Gizleme ve Güvenlik Riski: Serialization

Bir nesnenin diskte saklanması veya ağ üzerinden gönderilmesi (Serialization) sırasında, private veriler bile dışarıya sızabilir. NTP dilleri, serialization sırasında private alanların nasıl ele alınacağını kontrol etme mekanizmalarına sahiptir (örneğin Java'da 'transient' anahtar kelimesi). Güvenlik amaçlı private verilerin serialized edilmemesi gerekir.

Kapsülleme ve Test Edilebilirlik

İyi kapsüllenmiş sınıflar, genellikle daha küçük sorumluluklara sahiptir (SRP'ye uyarlar).

Private metotlar ve nitelikler, sınıfın iç mantığını dışarıdan soyutladığı için, sadece public arayüz (metotlar) test edilerek sınıfın doğru çalıştığı doğrulanabilir.

Bu, bağımsız birim testleri (unit testing) yazmayı kolaylaştırır.



Çok Biçimlilik ve Test Edilebilirlik

Çok biçimlilik sayesinde, bir sınıfın bağlı olduğu dış servisleri (veritabanı veya ağ bağlantıları) gerçekte çalıştırmadan test edebiliriz.

Bunu yapmak için, gerçek servisin arayüzünü uygulayan Sahte Nesneler (Mocks veya Stubs) kullanılır.

Bu sahte nesneler, test sırasında beklenen davranışı sergileyerek gerçek bağımlılıklara ihtiyaç duymadan test yapmayı sağlar.



Sonuç: NTP'nin Temel Direkleri

Kapsülleme, veri bütünlüğünü ve gizliliğini sağlayarak yazılımın güvenilirliğini artırır.

Çok Biçimlilik, kodun genişletilebilirliğini, esnekliğini ve yeniden kullanılabilirliğini maksimize eder.

Bu ilkeler, modern ve güvenli yazılım sistemlerinin tasarımında vazgeçilmezdir.



Kapsülleme ve Çok Biçimlilik

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

